

Personal Health Companion

Smart India Hackathon 2026 — Problem Statement SIH26181 (Qualcomm Inc) Theme: MedTech / BioTech / HealthTech · Category: Hardware

A privacy-preserving health companion that monitors a wearer's vitals in real time, cross-references them against environmental conditions, and raises early warnings — with emphasis on resilience during heatwaves, floods and pollution events.

What this prototype does

Senses	Heart rate + SpO ₂ (MAX30102), ambient temperature + humidity (DHT22), motion and falls (MPU6050), all on an ESP32
Detects	Tachycardia, bradycardia, hypoxia, falls, heat stress, poor air quality
Warns	Live dashboard banner → browser notification → SMS to an emergency contact for critical alerts
Correlates	Physiological strain against apparent temperature and AQI — the environmental cross-reference
Learns	An on-device autoencoder trained on real ICU patient vitals scores each 30 s window against normal physiology
Runs	Installable PWA, works offline against cached readings

The three things that go past plain thresholds

Heat index, not dry-bulb temperature. A naive rule (`temp > 40 && humidity > 60`) misses the physiology: **38 °C at 80 % RH is more dangerous than 42 °C at 20 % RH**, because sweat cannot evaporate. The backend computes the NOAA/NWS Rothfusz heat index and bands it (Safe → Caution → Extreme caution → Danger → Extreme danger). Verified against NOAA's published table: 100 °F at 65 % RH → 135.9 °F (table says 136 °F). Values are clamped at 58 °C, the ceiling of the published chart, and flagged when the inputs fall outside the regression's fitted range.

A personalised baseline. The engine learns each wearer's resting heart rate from their own at-rest samples (slow EWMA, guarded against PPG glitches) and expresses strain as a fraction of *their* reserve, rather than comparing everyone to the same number. Heat-stress alerts require the environment to be dangerous **and** the body to be responding to it.

A model trained on real patients. An autoencoder learns healthy physiology from the BIDMC dataset (PhysioNet — real HR and SpO₂ from ICU patients) and scores how far each 30-second window sits from normal. Splits are subject-wise and stratified, the threshold is chosen on validation, and inference is ~25 lines of dependency-free JavaScript so it runs in the browser with no cloud call and no WASM download. It answers *"this does not look like your normal"* — never *"you have condition X."*

Quick start

Two terminals. No hardware and no credentials needed — the simulator stands in for the ESP32, and SMS defaults to a console dry-run.

```
# terminal 1 – backend
cd backend
npm install
cp .env.example .env
npm run dev                # http://localhost:4000

# terminal 2 – frontend
cd frontend
npm install --legacy-peer-deps
cp .env.example .env
npm run dev                # http://localhost:3000
```

Then, in a third terminal, feed it data:

```
cd backend
npm run simulate            # steady normal vitals
npm run simulate:demo      # the full scripted demo sequence
```

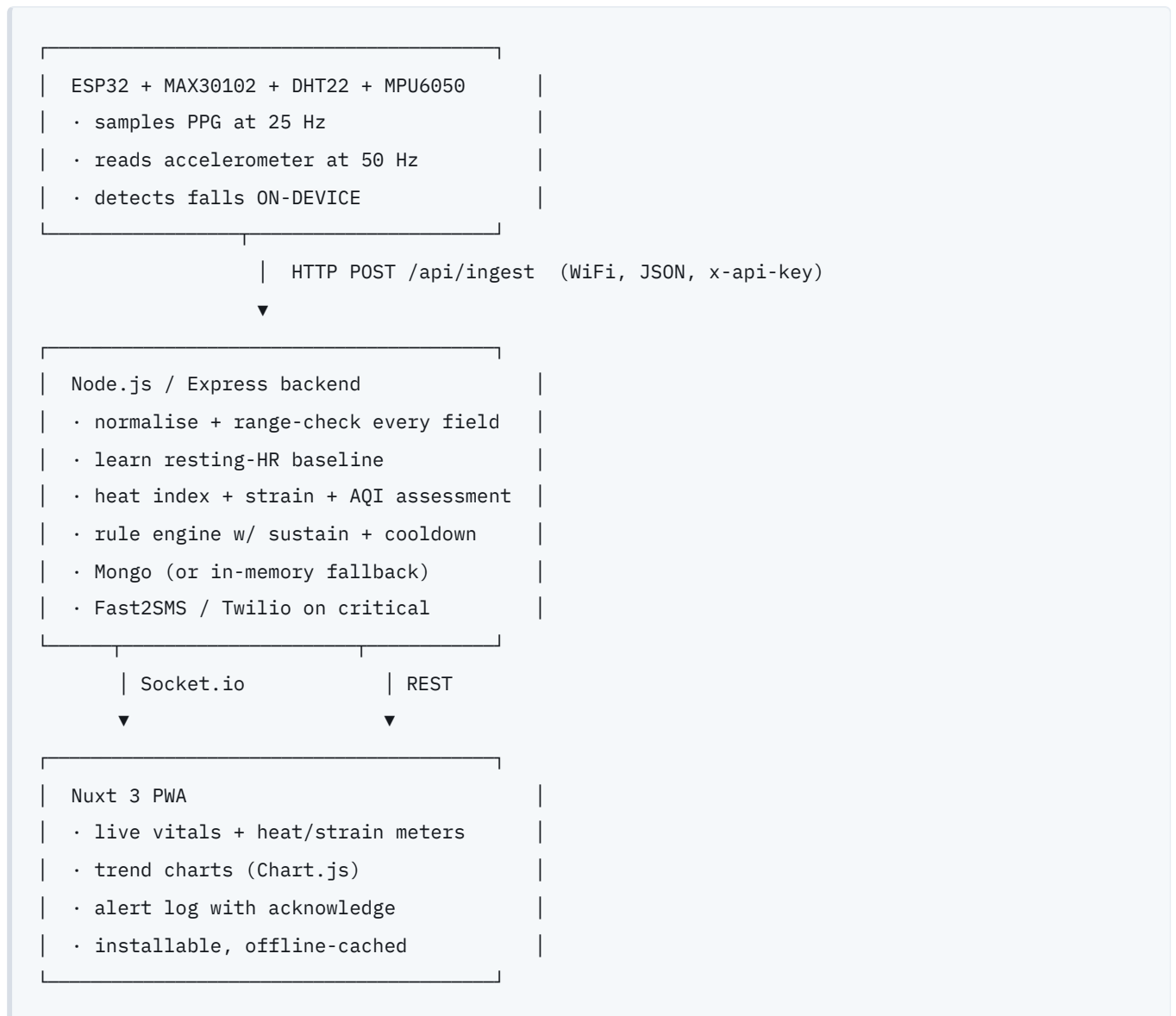
--legacy-peer-deps on the frontend is required, not optional. npm 10.9.x crashes with `Cannot read properties of null (reading 'edgesOut')` while resolving Nuxt's peer tree. The flag skips the code path that crashes; the install is otherwise identical.

Optional configuration

Everything below is optional — the app runs fully without any of it.

.ENV KEY	EFFECT WHEN SET
MONGO_URI	Persists readings and alerts to MongoDB Atlas. Blank = in-memory store , which still works end to end but loses data on restart.
OPENWEATHER_API_KEY	Adds live AQI / PM2.5 and a weather fallback when the device has no DHT22.
SMS_PROVIDER	<code>console</code> (default, prints to terminal) · <code>fast2sms</code> · <code>twilio</code>
EMERGENCY_CONTACT	Number that receives critical alerts.
DEVICE_API_KEY	Shared secret the ESP32 must present. Blank disables the check.

Architecture



Fall detection lives on the ESP32 rather than the server because it needs the accelerometer at ~50 Hz, and streaming 50 samples/second over WiFi would flatten the battery. The board decides; the server sees the verdict.

API

METHOD	ROUTE	PURPOSE
POST	/api/ingest	Sensor sample from the device
GET	/api/history/:deviceId? minutes=60&limit=600	Readings for the trend charts
GET	/api/alerts/:deviceId	Alert log
POST	/api/alerts/:alertId/ack	Acknowledge an alert
GET	/api/devices · /api/devices/:id	Registered wearers, thresholds, learned baseline
PUT	/api/devices/:id	Set wearer name, vulnerability profile, emergency contact, location
GET	/api/health	Liveness + which store/SMS/weather mode is active

Demo controls (stage scaffolding, isolated in `routes/demo.js`):

METHOD	ROUTE	PURPOSE
POST	/api/demo/env	Force ambient conditions. Body <code>{"preset": "heatwave"}</code> — also <code>pollution</code> , <code>flood</code> , <code>normal</code>
DELETE	/api/demo/env	Back to real weather
POST	/api/demo/reset/:deviceId	Clear baseline, sustain timers and cooldowns so a demo can be re-run immediately

Socket.io events: `reading:new`, `alert:triggered`, `alert:updated`, `alert:ack`, `device:status`.
Clients `emit('subscribe', deviceId)` to join that wearer's room.

`alert:triggered` fires the instant a rule matches; `alert:updated` follows a moment later carrying the SMS delivery outcome. The alert is never made to wait on a third-party SMS API.

Sample payload

```
{
  "deviceId": "band-001",
  "heartRate": 128, "spo2": 95, "bodyTemp": 37.2,
  "ambientTemp": 44, "humidity": 68,
  "accelMagnitude": 1.6, "motion": "active",
  "fallDetected": false, "signalOk": true,
  "timestamp": "2026-09-15T09:20:00Z"
}
```

Every vital is optional. A device with a flaky MAX30102 still reports temperature; out-of-range values become `null` rather than rejecting the whole sample.

Detection rules

ALERT	SEVERITY	CONDITION
hypoxia	critical	SpO ₂ below the profile's critical bound — fires immediately
hypoxia	critical	SpO ₂ below the warning bound, held 30 s
fall	critical	Free fall → impact → 2 s of stillness (decided on-device)
tachycardia	warning	HR above the profile bound, held 30 s
bradycardia	warning	HR below the profile bound, held 30 s
heat_stress	warning/critical	Heat-index band is dangerous and strain ≥ 45 % of reserve
air_quality	warning	OpenWeather AQI 5, or PM2.5 > 120 µg/m ³
ml_anomaly	warning	Learned model reconstruction error ≥ 1.6× threshold, held 30 s

The learned model **never suppresses a rule** — it only adds a signal. It is also optional: a clone with no `backend/src/ml/model.json` boots, runs every rule, and reports `mlScore: null`. See [ml/README.md](#) for training, methodology and measured results (**ROC AUC 0.807** on held-out subjects).

Thresholds tighten for the vulnerable groups the problem statement names:

PROFILE	HR HIGH	HR LOW	SPO ₂ WARN	SPO ₂ CRITICAL
general	120	50	92	88
elderly	110	45	93	89
outdoor_worker	125	45	92	88
chronic_condition	110	50	94	90

The 30-second sustain window exists so a single motion-corrupted PPG sample cannot page a caregiver. Alerts have a 60 s per-type cooldown and SMS a 5-minute one. When an SMS is suppressed by that cooldown the alert records why (`smsNote: "cooldown, 239s remaining"`) and the UI shows it — a held SMS never looks like a silent failure.

Demo script

Run `npm run simulate:demo` with the dashboard open. The sequence is timed to be recorded in one take:

1. **Baseline (20 s)** — wearer at rest, vitals normal, the app learns their resting HR.
2. **Exertion (40 s)** — HR climbs past threshold, holds 30 s, `tachycardia` warning appears.
3. **Heatwave (35 s)** — ambient pushed to 44 °C; the heat-index meter goes red and `heat_stress` fires.
This is the environmental cross-reference — say so out loud.
4. **Desaturation (25 s)** — SpO₂ falls below 88 %, **critical** alert, SMS dispatched.
5. **Fall (12 s)** — impact then stillness, **critical** alert, SMS dispatched.

With real hardware, replace steps 2 and 3 with a wearer doing jumping jacks and a hair dryer near the DHT22. Step 4 cannot be staged safely on a person — keep it simulated, and say that it is.

To show the SMS without credentials, keep `SMS_PROVIDER=console` and put the backend terminal on screen. For a real SMS, set `SMS_PROVIDER=fast2sms` and `FAST2SMS_API_KEY`.

Layout

```
health-companion/
├─ backend/
│  └─ src/
│     ├─ models/           Reading, Alert, Device (Mongoose schemas)
│     ├─ routes/           ingest, history, alerts, devices, demo
│     ├─ services/         anomalyDetection, heatStress, weatherService, smsService, mlDetector
│     ├─ ml/               anomalyModel.js (inference) + model.json (weights)
│     ├─ store.js          Mongo ⇌ in-memory facade
│     ├─ socket.js         Socket.io rooms and emitters
│     ├─ simulator.js      stands in for the ESP32
│     └─ server.js
├─ frontend/
│  ├─ pages/               index (live) · history (trends) · alerts (log)
│  ├─ components/         VitalTile · AlertBanner · TrendChart · SparkLine
│  ├─ composables/        useHealth.ts – socket + shared state
│  └─ nuxt.config.ts       PWA manifest, workbox caching
├─ ml/                    training pipeline (PyTorch) – see ml/README.md
│  ├─ prepare_data.py      BIDMC numerics → windows, subject-wise split
│  ├─ train.py             autoencoder + evaluation
│  ├─ diagnose_fit.py      capacity sweep, over/under-fit evidence
│  └─ export_weights.py    → backend/src/ml/model.json
├─ docs/screenshots/      captures of the running app
└─ firmware/
   ├─ BRINGUP.md           beginner hardware guide
   ├─ bringup/             t1-t7 staged test sketches
   └─ esp32_sensor_node/   .ino + config.h (gitignored) + config.example.h
```

Firmware

New to hardware? Start with [firmware/BRINGUP.md](#) — a step-by-step guide written for someone who has never wired a breadboard: what to buy, how to wire it, and seven staged test sketches in [firmware/bringup/](#) that prove one thing each (board alive → WiFi → I²C wiring → each sensor → backend reachable) before you run the real firmware.

Copy `config.example.h` to `config.h` and set your WiFi credentials and your laptop's **LAN IP** (not `localhost` — the ESP32 has to reach it across the network).

Arduino libraries required:

- SparkFun MAX3010x Pulse and Proximity Sensor Library
- DHT sensor library (Adafruit) + Adafruit Unified Sensor
- Adafruit MPU6050

Wiring — MAX30102 and MPU6050 share I2C on GPIO21 (SDA) / GPIO22 (SCL); DHT22 data on GPIO4 with a 10 kΩ pull-up.

Deliberately out of scope for this stage

Named here so they read as roadmap rather than as gaps:

- **Inference on the MCU itself.** The learned detector runs in the app today; porting a quantised model onto the ESP32 is the next step, and the JSON weights were chosen partly to keep that path open.
- **Offline BLE mesh / LoRa relay** for SOS when the wearer's phone has no signal.
- **Real-time multilingual voice alerts.** A pre-recorded clip is enough for the demo.

Known limitations

- MAX30102 SpO₂ is unreliable under motion; the firmware sets `signalOk:false` when no finger is detected and the dashboard greys those tiles out, but wrist-worn PPG during activity is genuinely hard.
- DHT22 measures temperature and humidity only — **it cannot measure PM2.5**. Air-quality alerts need either the OpenWeather API or a real particulate sensor (PMS5003 / SPS30).
- The in-memory store caps at 5000 readings and 500 alerts, and clears on restart.
- **The app cannot run on a phone as configured.** `NEXT_PUBLIC_API_BASE` points at `localhost`, and service workers need a secure context — a LAN IP over plain HTTP will not register one, so there is no install prompt and no offline. Needs HTTPS on both ends (a tunnel, or deployment).
- **Browser notifications do not fire on Android.** `useHealth.ts` uses `new Notification(...)`, which Android Chrome rejects for installed PWAs; it needs `ServiceWorkerRegistration.showNotification()`. The throw is caught, so it fails silently.
- The learned model's standalone recall is low (0.32) and one atypical-but-healthy test subject drives most of its false positives. It is a complementary signal, not the detector.